Mayan Sui

Security Assessment

March 4th, 2025 — Prepared by OtterSec

Akash Gurugunti sud0u53r.ak@osec.io

Ajay Shankar Kunapareddy d1r3wolf@osec.io

Robert Chen r@osec.io

Table of Contents

Executive Summary **2**

Overview 2

Key Findings 2

Scope **3**

Findings **4**

Vulnerabilities **5**

OS-MSI-ADV-00 | Permissionless Order Fulfillment Before Penalty Period 6

OS-MSI-ADV-01 | Loss of Funds Due to Invalid Gas Recipient 7

OS-MSI-ADV-02 | Missing Version Checks in Admin Functions 8

General Findings **9**

OS-MSI-SUG-00 | Validation and Consistency Checks 10

OS-MSI-SUG-01 | Missing Validation Logic 11

OS-MSI-SUG-02 | Code Maturity 12

OS-MSI-SUG-03 | Redundant/Unutilized Code 14

Appendices

Vulnerability Rating Scale **15**

Procedure **16**

© 2025 Otter Audits LLC. All Rights Reserved.

01 — Executive Summary

Overview

Mayan Finance engaged OtterSec to assess the `swift-sui` program. This assessment was conducted between January 10th, 2024 and February 14th, 2025. For more information on our auditing methodology, refer to Appendix B.

Key Findings

We produced 7 findings throughout this audit engagement.

In particular, we identified a vulnerability concerning improper access control in the order fulfillment functionality, where the driver check on the sender is only enforced during the penalty period, allowing anyone to fulfill the order before the penalty period begins (OS-MSI-ADV-00). Additionally, there is a lack of version checks in the admin functions, allowing the utilization of outdated package versions and enabling the bypassing of restrictions and security measures in newer implementations (OS-MSI-ADV-02). Furthermore, we highlighted a potential loss of funds issue, where sending gas to the destination address in the case of a custom payload results in irretrievable funds, as object IDs are unable to receive coins (OS-MSI-ADV-01).

We also made recommendations regarding the need for improved validation and consistency checks (OS-MSI-SUG-01), and suggested removing unnecessary and redundant instances (OS-MSI-SUG-03). Moreover, we advised adherence to coding best practices (OS-MSI-SUG-02).

© 2025 Otter Audits LLC. All Rights Reserved.

02 — Scope

The source code was delivered to us in a Git repository at https://github.com/mayan-finance/mayan_swift_sui. This audit was performed against commit f0d914a.

A brief description of the program is as follows:

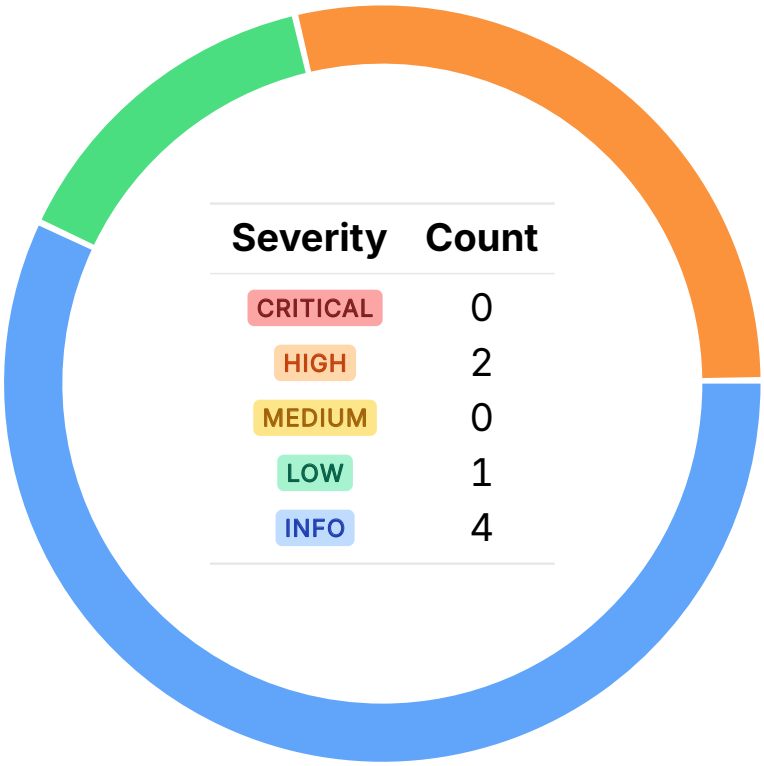
Name	Description
swift-sui	A cross-chain token transfer and order fulfillment protocol implemented on Sui that uses Wormhole for secure messaging between blockchains.

© 2025 Otter Audits LLC. All Rights Reserved.

03 — Findings

Overall, we reported 7 findings.

We split the findings into **vulnerabilities** and **general findings**. Vulnerabilities have an immediate impact and should be remediated as soon as possible. General findings do not have an immediate impact but will aid in mitigating future vulnerabilities.



Severity	Count
CRITICAL	0
HIGH	2
MEDIUM	0
LOW	1
INFO	4

© 2025 Otter Audits LLC. All Rights Reserved.

04 — Vulnerabilities

Here, we present a technical analysis of the vulnerabilities we identified during our audit. These vulnerabilities have *immediate* security implications, and we recommend remediation as soon as possible.

Rating criteria can be found in Appendix A.

ID	Severity	Status	Description
OS-MSI-ADV-00	HIGH	RESOLVED	The check `msg_driver == ctx.sender` is enforced only during the penalty period, allowing anyone to fulfill the order before the penalty period begins.
OS-MSI-ADV-01	HIGH	RESOLVED	If `PAYLOAD_TYPE_CUSTOM_PAYLOAD` is utilized, `addr_dest` points to a `DestCap` object ID, not a user address. Sending gas to it results in irretrievable funds, as object IDs will not be able to receive coins.
OS-MSI-ADV-02	LOW	RESOLVED	Lack of version checks in admin functions allows the utilization of outdated package versions, enabling the bypassing of restrictions and security measures in newer implementations.

© 2025 Otter Audits LLC. All Rights Reserved.

Permissionless Order Fulfillment Before Penalty Period HIGH OS-MSI-ADV-00

Description

`fulfill::prepare_fulfill_winner` lacks proper access control checks, resulting in the unauthorized fulfillment of orders. The function currently enforces `msg_driver == ctx.sender` only during the penalty period, which is defined as the time window between `order_item.deadline() - penalty_period` and `order_item.deadline`.

>_ mayan_swift/sources/fulfill.move RUST

```rust
public fun prepare_fulfill_winner([...]): FulfillTicket {
    [...]
    let clock_now_s = clock.timestamp_ms() / 1000;
    assert!(order_item.deadline() > clock_now_s, EDeadlineIsPassed);

    if (clock_now_s >= order_item.deadline() - (order_item.penalty_period() as u64)) {
        assert!(msg_driver == ctx.sender(), EInvalidDriver);
    };

    [...]
}
```

Before the penalty period, there is no restriction on who may call the function. Anyone may prepare the fulfillment before the penalty period, regardless of whether they are the legitimate auction winner (`msg_driver`).

Remediation

Enforce the driver check before the penalty period condition to ensure that only the legitimate auction winner may prepare the fulfillment before the penalty period.

Patch

Resolved in ac8a875.

© 2025 Otter Audits LLC. All Rights Reserved.

Loss of Funds Due to Invalid Gas Recipient HIGH OS-MSI-ADV-01

Description

In `PAYLOAD_TYPE_CUSTOM_PAYLOAD` mode, the function incorrectly assumes that `addr_dest` is a user address and sends the gas coin to it. However, `addr_dest` actually refers to the ID of a `DestCap`, which is not a real account. As a result, the sent gas coin will be irretrievable, since `DestCap` cannot receive funds.

>_ *mayan_swift_sui/packages/mayan_swift/sources/state.move* RUST

```rust
public(package) fun fulfill_dest_order<CoinType>(
     state: &mut State,
     fund: Coin<CoinType>,
     gas: Coin<SUI>,
     addr_dest: address,
     [...]
     payload_type: u8,
     custom_payload: address,
     ctx: &mut TxContext,
): (u8, u64, Option<ID>) {
    if (gas.value() > 0) {
        transfer::public_transfer(gas, addr_dest);
    } else {
        gas.destroy_zero();
    };
    [...]
}
```

Remediation

Refrain from sending gas to `addr_dest` if the payload type is custom.

Patch

Resolved in 95ce7b8.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Version Checks in Admin Functions LOW OS-MSI-ADV-02

Description

Currently, in `mayan-swift`, admin functions are missing version checks. In `fee_manager::state`, admin functions such as `add_token_config`, `update_token_config`, and `update_fee_collector` do not include any version checks, which implies that an administrator may utilize an older version of the contract to bypass restrictions that are enforced in newer versions. Similarly, version checks are not present in `change_paused` and `add_pair_emitter` in `mayan_swift::state`.

Remediation

Ensure to update admin functions with checks to verify the utilized object is the supported version.

Patch

Resolved in ac8a875.

© 2025 Otter Audits LLC. All Rights Reserved.

05 — General Findings

Here, we present a discussion of general findings during our audit. While these findings do not present an immediate security impact, they represent anti-patterns and may result in security issues in the future.

ID	Description
OS-MSI-SUG-00	`initialize_order` lacks critical validation checks for `chain_dest`, `input_funds`, and `auction_mode`, risking invalid transactions and inconsistent state management.
OS-MSI-SUG-01	There are several instances where proper validation is not performed, resulting in potential security issues.
OS-MSI-SUG-02	Suggestions regarding inconsistencies in the codebase and ensuring adherence to coding best practices.
OS-MSI-SUG-03	The codebase contains multiple cases of redundant and unnecessary code that should be removed for better maintainability and clarity.

© 2025 Otter Audits LLC. All Rights Reserved.

Validation and Consistency Checks OS-MSI-SUG-00

Description

`init_order::initialize_order` is designed for cross-chain swaps but currently lacks several validation and consistency checks. Specifically, the `chain_dest` parameter specifies the target blockchain for the transaction, while `wormhole::state::chain_id` returns the current (source) chain ID. If `chain_dest` is the same as the source chain, it will result in a self-chain transfer, which does not align with the intended cross-chain behavior.

Also, it is possible for an order to have zero input funds, which does not make sense for a swap operation. Lastly, there is no validation to ensure that `auction_mode` is a valid value.

Remediation

Refactor `initialize_order` to ensure that `chain_dest != wormhole::state::chain_id`, `input_funds.value() > 0`, and `auction_mode` is a valid value.

Patch

Resolved in ac8a875.

© 2025 Otter Audits LLC. All Rights Reserved.

Missing Validation Logic

OS-MSI-SUG-01

Description

1. Sui and Solana currently handle order deadline checks differently in `cancel_order`. Sui treats the deadline as inclusive (`now >= deadline`), while Solana utilizes an exclusive check (`now > deadline`). This inconsistency may result in cross-chain behavior mismatches and failed cancellations. Standardize this logic across chains, ideally utilizing an inclusive check for clarity and consistency.
2. Add a check in the following functions to ensure that the current version is valid and not outdated or deprecated:
   - `init_order::prepare_initialize_order`
   - `init_order::log_initialize_order`
   - `settle::complete_settle`
   - `unlock_batch::complete_unlock_batch`

Remediation

Incorporate the above valuations into the codebase.

Patch

Issue #1 resolved in 95ce7b8.

© 2025 Otter Audits LLC. All Rights Reserved.

Code Maturity

OS-MSI-SUG-02

Description

1. Define a custom error when checking that the `VecSet` is not empty in `max_compatible_state_version` within `fee_manager::state` and `mayan_swift::state`.

>_ sources/state.move RUST

```rust
fun max_compatible_state_version(state: &State): u64 {
    let keys = state.version_set.keys();
    let len = vector::length(keys);
    assert!(len > 0, 1); // Ensure the VecSet is not empty
    let mut max_value = *vector::borrow(keys, 0); // Initialize with the first element
    let mut i = 1;
    while (i < len) {
        let current = *vector::borrow(keys, i);
        if (current > max_value) {
            max_value = current;
        };
        i = i + 1;
    };
    max_value
}
```

2. `post_batch` emits a `BatchUnlockPosted` event, which tracks which orders were posted and whether the message is compact. However, the `compact` field is hardcoded to false, regardless of the actual `compact` value passed as a parameter to `post_batch`. This implies that even if the batch message was actually serialized in compact format, the emitted `BatchUnlockPosted` event incorrectly states that it was not. Ensure that the event emission utilizes the actual `compact` parameter rather than emitting a hardcoded value.

>_ sources/batch_post.move RUST

```rust
public fun post_batch(
    state: &mut State,
    orders: vector<address>,
    compact: bool,
): MessageTicket {
    [...]
    sui::event::emit(BatchUnlockPosted {
        compact: false,
        orders,
    });
    message_ticket
}
```

© 2025 Otter Audits LLC. All Rights Reserved.

3. In `refund::refund`, fix the typographical error where the `coin_metadata` parameter is misspelled as `cion_metadata`.
4. Extract common logic from `mayan_swift::unlock::unlock` and `mayan_swift::unlock_batch::unlock_batch_item`, such as validation checks and order unlocking calls, into a shared function to improve code re-usability, maintainability, and efficiency.

Remediation

Implement the above-mentioned suggestions.

Patch

Issue #2 resolved in 95ce7b8.

© 2025 Otter Audits LLC. All Rights Reserved.

Redundant/Unutilized Code

OS-MSI-SUG-03

Description

1. The `input_funds.value() >= amount_in_min` check in `prepare_initialize_order` within `init_order` is unnecessary and may be removed.
2. In `state::verify_auction_emitter`, the `chain_id != wormhole::state::chain_id` check is redundant since `chain_id` is already verified in the following `chain_id == SOLANA_CHAIN_ID` check.

   >_ *sources/state.move* RUST

   ```rust
   public(package) fun verify_auction_emitter(state: &State, emitter_address: address,
       ↪  chain_id: u16) {
       assert!(chain_id != wormhole::state::chain_id(), EInvalidEmitter);
       assert!(emitter_address == state.emitter_paired.auction, EInvalidEmitter);
       assert!(chain_id == SOLANA_CHAIN_ID, EInvalidEmitter);
   }
   ```

3. `token_configs` and `latest_package_id` in `Fee_manager::state` seem unnecessary and may be removed.

Remediation

Remove the redundant/unnecessary code instances listed above.

Patch

Issue #1 resolved in ac8a875.

© 2025 Otter Audits LLC. All Rights Reserved.

A — Vulnerability Rating Scale

We rated our findings according to the following scale. Vulnerabilities have immediate security implications. Informational findings may be found in the General Findings.

CRITICAL

Vulnerabilities that immediately result in a loss of user funds with minimal preconditions.

Examples:

- Misconfigured authority or access control validation.
- Improperly designed economic incentives leading to loss of funds.

HIGH

Vulnerabilities that may result in a loss of user funds but are potentially difficult to exploit.

Examples:

- Loss of funds requiring specific victim interactions.
- Exploitation involving high capital requirement with respect to payout.

MEDIUM

Vulnerabilities that may result in denial of service scenarios or degraded usability.

Examples:

- Computational limit exhaustion through malicious input.
- Forced exceptions in the normal user flow.

LOW

Low probability vulnerabilities, which are still exploitable but require extenuating circumstances or undue risk.

Examples:

- Oracle manipulation with large capital requirements and multiple transactions.

INFO

Best practices to mitigate future security risks. These are classified as general findings.

Examples:

- Explicit assertion of critical internal invariants.
- Improved input validation.

© 2025 Otter Audits LLC. All Rights Reserved.

B — Procedure

As part of our standard auditing procedure, we split our analysis into two main sections: design and implementation.

When auditing the design of a program, we aim to ensure that the overall economic architecture is sound in the context of an on-chain program. In other words, there is no way to steal funds or deny service, ignoring any chain-specific quirks. This usually requires a deep understanding of the program's internal interactions, potential game theory implications, and general on-chain execution primitives.

One example of a design vulnerability would be an on-chain oracle that could be manipulated by flash loans or large deposits. Such a design would generally be unsound regardless of which chain the oracle is deployed on.

On the other hand, auditing the program's implementation requires a deep understanding of the chain's execution model. While this varies from chain to chain, some common implementation vulnerabilities include reentrancy, account ownership issues, arithmetic overflows, and rounding bugs.

As a general rule of thumb, implementation vulnerabilities tend to be more "checklist" style. In contrast, design vulnerabilities require a strong understanding of the underlying system and the various interactions: both with the user and cross-program.

As we approach any new target, we strive to comprehensively understand the program first. In our audits, we always approach targets with a team of auditors. This allows us to share thoughts and collaborate, picking up on details that others may have missed.

While sometimes the line between design and implementation can be blurry, we hope this gives some insight into our auditing procedure and thought process.

© 2025 Otter Audits LLC. All Rights Reserved.